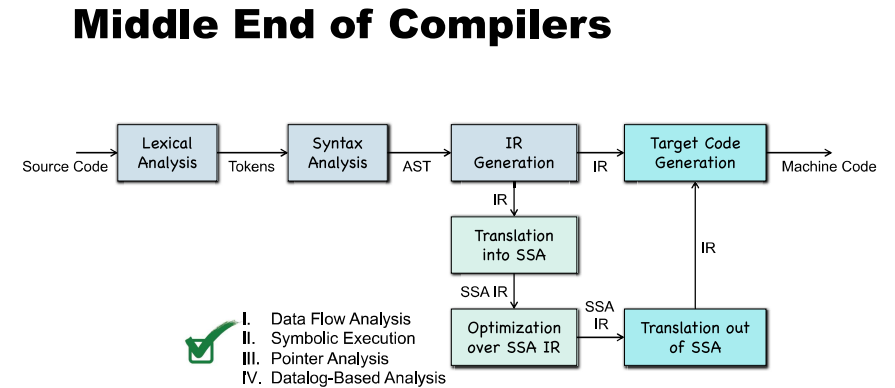
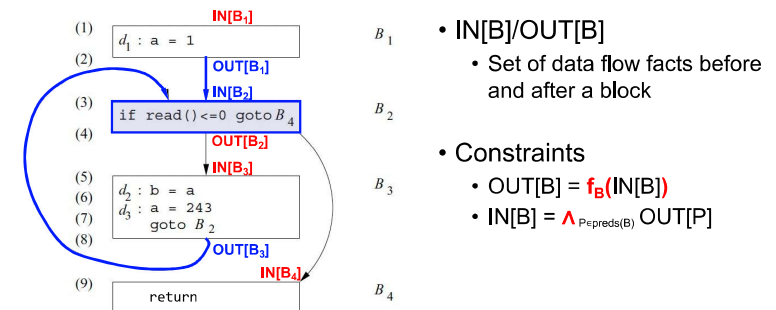


Recap-2 The Compilers' Middle End



2

Data Flow Analysis



5

PART I: Data Flow Analysis

Data Flow Analysis

- To perform data flow analysis, define
 - transfer function f for each statement/block
 - merge function $\bigwedge$ at the joint point of multiple paths
 - the **initial** set of data flow facts for each block
- Data flow analysis produces
 - $IN[B]/OUT[B]$ that include data flow facts at a program point
 - The resulting data flow facts are always true during program execution

7

The Worklist Algorithm

- **ForEach** basic block B : initialize $IN[B]$ and $OUT[B]$; **EndFor**
- $worklist =$ set of all basic blocks;
- **While** ($!worklist.empty()$) **Do**
 - $B = worklist.pop()$;
 - $IN[B] = \bigwedge_{P \in preds(B)} OUT[P]$; $OUT[B] = f_B(IN[B])$
 - **If** $OUT[B]$ changed: $worklist.push$ (all B 's successors); **EndIf**
- **EndWhile**

8

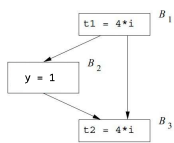
Classic DFA

- Reaching Definition
- Available Expressions
- Live Variables

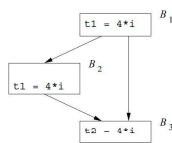
9

Available Expressions

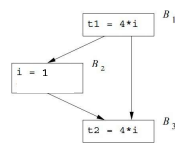
- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .



$4 * i$ is available before B_3



$4 * i$ is available before B_3



$4 * i$ is **NOT** available before B_3

11

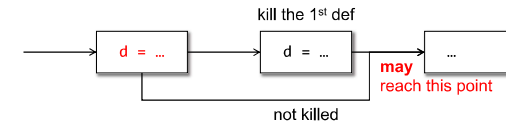
Transfer & Merging Functions

- Transfer Functions?
- Merge Functions?
- Forward or Backward Analysis?

14

Reaching Definitions

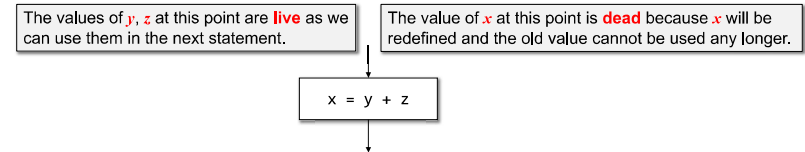
- A definition d may reach a program point, if there is a path from d to the program point such that d is not killed along the path.



10

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .



13

PART II: Symbolic Execution

15

Symbolic Execution

- Path Sensitivity
- Flow Sensitivity
- Context Sensitivity

16

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - Unsatisfiable
 - Unknown (due to timeout)



18

The SAT Problem

- Deciding satisfiability of a formula F in PL
- **Step 1:** Transforming F to CNF by Tseitin Transformation
- **Step 2:** Invoke the DPLL algorithm

What is CNF?
Why not DNF or NNF?

22

Symbolic Execution

- Path Explosion
- Constraint Solving
- External Function Call

17

Solving Path Constraints

- Propositional Logic
- Satisfiability (SAT) --- DPLL and CDCL

-
- First Order Logic
 - Satisfiability Modulo Theories (SMT)
 - The Bit Vector Theory

19

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- **Goal:** Find a solution to satisfy the input formula
- Repeat the following steps:
 - (1) Choose a variable, p , assign true or false to the variable
 - (2) Propagate the value of p

24

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```
bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
```

25

DPLL

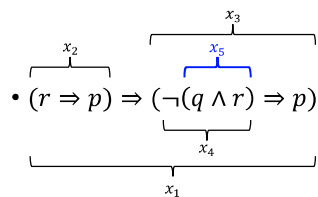
- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee q) \wedge (p \vee \neg q) \wedge \neg p$
- $p \wedge \neg p$ **False! Unsatisfiable!**

Theorem [Resolution]

$(a_0 \vee a_1 \vee \dots \vee a_n \vee c) \wedge (b_0 \vee b_1 \vee \dots \vee b_m \vee \neg c)$
can be simplified into
 $(a_0 \vee a_1 \vee \dots \vee a_n \vee b_0 \vee b_1 \vee \dots \vee b_m)$

32

Tseitin Transformation



$x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3))$
 $\wedge x_2 \Leftrightarrow (r \Rightarrow p)$
 $\wedge x_3 \Leftrightarrow (x_4 \Rightarrow p)$
 $\wedge x_4 \Leftrightarrow \neg(x_5)$
 $\wedge x_5 \Leftrightarrow q \wedge r$

$x_4 \Rightarrow \neg x_5 \wedge \neg x_5 \Rightarrow x_4$
 $(\neg x_4 \vee \neg x_5) \wedge (x_5 \vee x_4)$

46

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```
bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

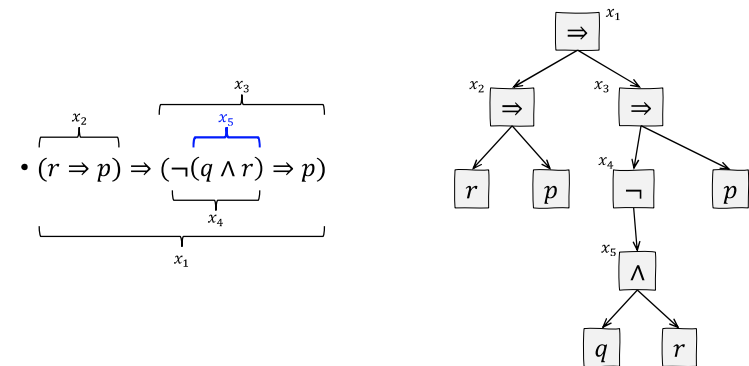
    return DPLL(F[p→false], I[p→ false]);
}
```

Optimizations may be applied!

$(a_0 \vee a_1 \vee \dots \vee a_n \vee c) \wedge (b_0 \vee b_1 \vee \dots \vee b_m \vee \neg c)$
can be simplified into
 $(a_0 \vee a_1 \vee \dots \vee a_n \vee b_0 \vee b_1 \vee \dots \vee b_m)$

26

Tseitin Transformation



38

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \dots, a_0]$, $y = [b_7, b_6, \dots, b_0]$ and $z = [c_7, c_6, \dots, c_0]$ are three 8bit bit vectors

$x \mid y = z \xrightarrow{\text{Step 2}} \bigwedge_{i=0}^7 ((a_i \vee b_i) \Leftrightarrow c_i)$

49

Andersen & Steensgaard Algorithm

Statement	Constraint	Shorthand
$y = \&x$	$\text{pts}(y) \supseteq \{x\}$	
$y = x$	$\text{pts}(y) \supseteq \text{pts}(x)$	
$*y = x$	$\forall v \in \text{pts}(y): \text{pts}(v) \supseteq \text{pts}(x)$	$\text{pts}(*y) \supseteq \text{pts}(x)$
$y = *x$	$\forall v \in \text{pts}(x): \text{pts}(y) \supseteq \text{pts}(v)$	$\text{pts}(y) \supseteq \text{pts}(*x)$

Statement	Constraint	Shorthand
$y = \&x$	$\text{pts}(y) \supseteq \{x\}$	
$y = x$	$\text{pts}(y) = \text{pts}(x)$	
$*y = x$	$\forall v \in \text{pts}(y): \text{pts}(v) = \text{pts}(x)$	$\text{pts}(*y) = \text{pts}(x)$
$y = *x$	$\forall v \in \text{pts}(x): \text{pts}(y) = \text{pts}(v)$	$\text{pts}(y) = \text{pts}(*x)$

51

- Each graph node x denotes the points-to set $pts(x)$
- Solving the set constraints via a dynamic transitive closure



PART IV: Datalog-Based Analysis

- **def(B, N, X)**
 - the Nth statement in **Block B** may define the variable **X**.



- ```

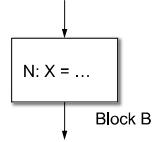
graph LR
 In(()) --> B1[M: X = ...]
 B1 --> B2[N: print X]
 B2 --> Out(())
 subgraph " "
 B1
 B2
 end
 B1 --- BC[Block C]
 B2 --- BB[Block B]

```

63

## Reaching Definitions by Datalog

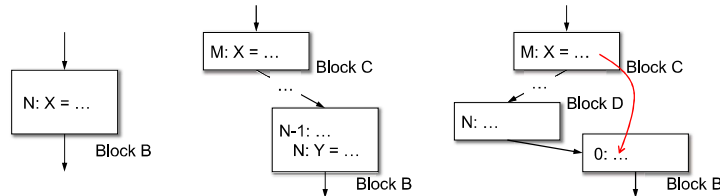
- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$
- $rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$
- $rd(B, 0, C, M, X) :- rd(D, N, C, M, X), \text{succ}(D, N, B)$



68

## Reaching Definitions by Datalog

- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$
- $rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$
- $rd(B, 0, C, M, X) :- rd(D, N, C, M, X), \text{succ}(D, N, B)$



70

## Reaching Definitions by Datalog

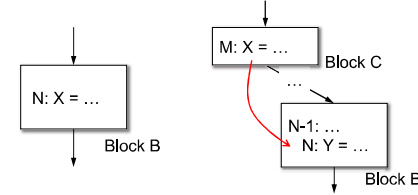
- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$
- $rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$
- $rd(B, 0, C, M, X) :- rd(D, N, C, M, X), \text{succ}(D, N, B)$

- $\text{def}(B_1, 1, i)$
- $\text{def}(B_2, 2, j)$
- $\text{succ}(B_2, 2, B_3)$
- $\text{def}(B_1, 2, j)$
- $\text{def}(B_3, 1, a)$
- $\text{succ}(B_2, 2, B_4)$
- $\text{def}(B_1, 3, a)$
- $\text{def}(B_4, 1, i)$
- $\text{succ}(B_3, 1, B_4)$
- $\text{def}(B_2, 1, i)$
- $\text{succ}(B_1, 3, B_2)$
- $\text{succ}(B_4, 1, B_2)$

74

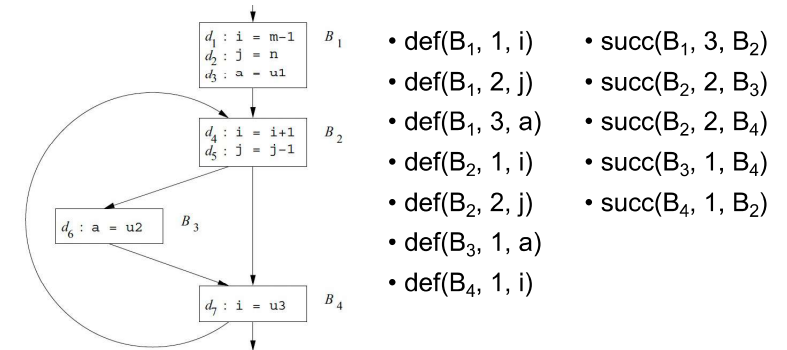
## Reaching Definitions by Datalog

- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$
- $rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$
- $rd(B, 0, C, M, X) :- rd(D, N, C, M, X), \text{succ}(D, N, B)$



69

## Reaching Definitions by Datalog



- $\text{def}(B_1, 1, i)$
- $\text{def}(B_1, 2, j)$
- $\text{def}(B_1, 3, a)$
- $\text{def}(B_2, 1, i)$
- $\text{def}(B_2, 2, j)$
- $\text{def}(B_3, 1, a)$
- $\text{def}(B_4, 1, i)$
- $\text{succ}(B_1, 3, B_2)$
- $\text{succ}(B_2, 2, B_3)$
- $\text{succ}(B_2, 2, B_4)$
- $\text{succ}(B_3, 1, B_4)$
- $\text{succ}(B_4, 1, B_2)$

73

## Reaching Definitions by Datalog

- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$

We just define facts and rules.  
Analysis is automatically done by Datalog engines!

- $\text{def}(B_1, 1, i)$
- $\text{def}(B_1, 2, j)$
- $\text{def}(B_1, 3, a)$
- $\text{def}(B_2, 1, i)$
- $\text{def}(B_2, 2, j)$
- $\text{def}(B_3, 1, a)$
- $\text{def}(B_4, 1, i)$
- $\text{succ}(B_1, 3, B_2)$
- $\text{succ}(B_2, 2, B_3)$
- $\text{succ}(B_2, 2, B_4)$
- $\text{succ}(B_3, 1, B_4)$
- $\text{succ}(B_4, 1, B_2)$

Query Example:  $rd(B_4, 1, B_1, 1, i)$

75